

Summer Project

Mahanth Reddy

Progress Report: Last 2-3 Months

Overview

Over the past few months, my core focus has been developing, processing, and evaluating a vision-based autonomous driving pipeline. This project bridges raw sensor data collected in the **CARLA Simulator (Unreal Engine 5)** with **MASt3R-Nav**, a state-of-the-art neural network designed for spatial navigation.

The work spanned large-scale dataset generation, complex coordinate space transformations (translating CARLA's left-handed space to ROS's right-handed space), and model benchmarking in an offline Open-Loop environment.

System Pipeline

The architecture of the implemented pipeline is illustrated below:

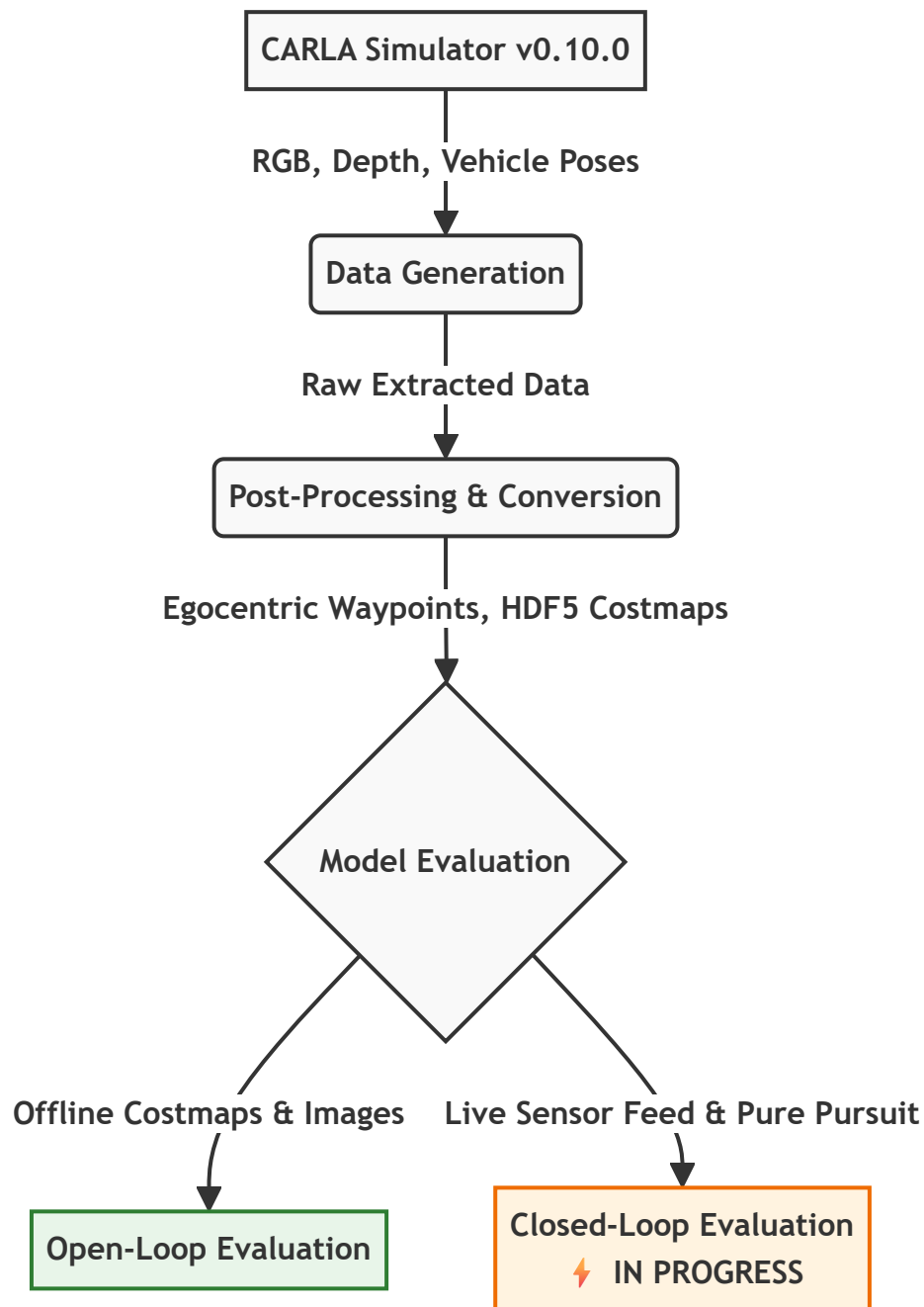


Figure 1: Data flow from synthetic generation to downstream evaluation.

Methodology & Implementation

1. Data Generation

Using CARLA, I generated diverse driving trajectories. The ego-vehicle captured synchronous sensor data, including 320x240 RGB images, metric depth maps, and precise agent states (x, y, z , roll, pitch, yaw). Additionally, ground-truth cost-to-goal maps were computed to provide the network with traversability context.



Figure 2a: Sample Egocentric RGB Camera View



Figure 2b: Corresponding Topographical Costmap

2. Format Conversion & Post-Processing

Because CARLA is built on a video game engine, its spatial logic is incompatible with standard robotics networks. I executed a conversion pipeline to:

Align coordinate systems by transforming CARLA's Left-Handed space into ROS Right-Handed space.

Apply an **egocentric transformation** so the network learns waypoints relative to its current camera view rather than absolute global GPS coordinates.

Pack tens of thousands of floating-point costmaps into an HDF5 format to prevent I/O bottlenecks during model execution.

3. Open-Loop Evaluation

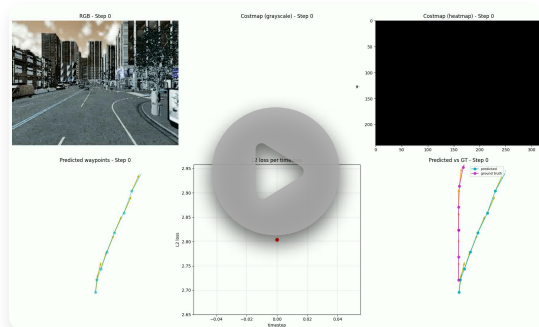
I benchmarked the trained **ObjRelLearntController** offline. The model acted as a "passenger," predicting physical (X, Y) waypoints based on pre-recorded images

and costmaps. We calculated the L2 distance between the network's predictions and the human Ground Truth trajectories.

Qualitative Results

Below are qualitative breakdowns for each evaluated trajectory, displaying the live open-loop evaluation video overlay alongside the corresponding per-timestep L2 loss curve.

Trajectory 1: `world00_traj_0_0`



Video 1: Click to view Open-Loop Overlay

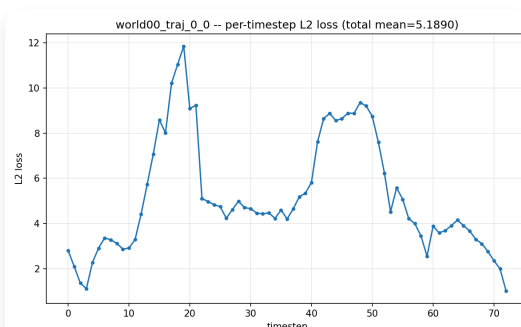
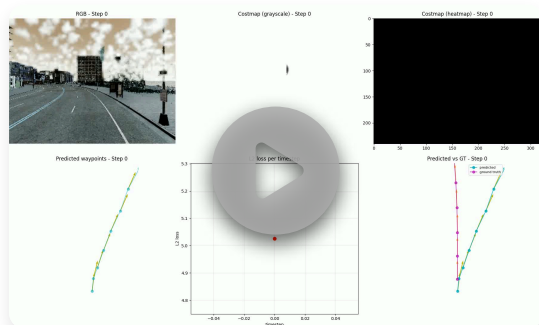


Figure 3a: Per-Timestep L2 Loss Curve

Trajectory 2: `world00_traj01`



Video 2: Click to view Open-Loop Overlay

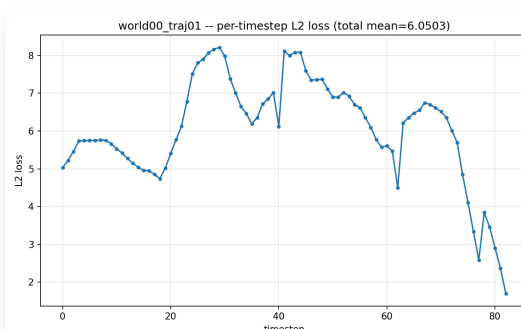


Figure 3b: Per-Timestep L2 Loss Curve

Quantitative Results

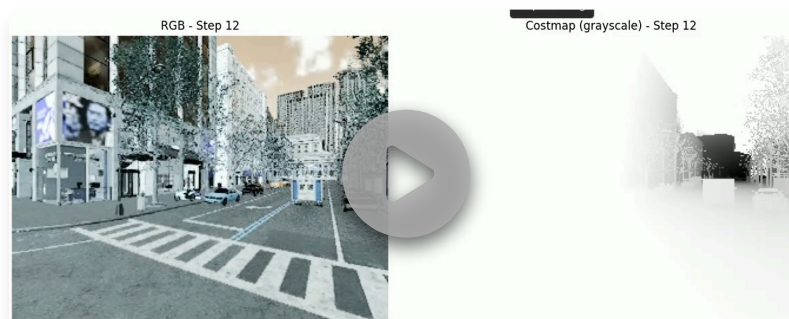
The table below outlines the open-loop trajectory metrics and total L2 mean loss errors recorded across evaluated scenes.

Trajectory Name	Horizon Length	Mean L2 Loss (Meters)
world00_traj_0_0	5 steps	5.1890 m
world00_traj01	5 steps	6.0503 m
Overall Mean Loss (Across 2 Trajectories)		5.6197 m

Limitations

Dataset Artifacts (Ego Collisions): A notable limitation encountered during data generation stems from CARLA's global route planner, which is optimized for dynamic traffic navigation rather than producing lengthy, continuously collision-free trajectories. Because this pipeline required extended trajectory lengths for the dataset, the ego vehicle occasionally collided with leading vehicles in its own lane.

Although a filtering pass was applied in an attempt to remove affected frames, this process was not entirely complete, leaving some residual collision data in the dataset. As a result, the trained controller sometimes erroneously predicts a forward path even when an obstacle is directly in front of it.



Video 3: Example of the controller attempting to drive straight despite a leading vehicle, due to dataset collision artifacts[cite: 1].

AI Tool Usage Declaration

In accordance with reporting guidelines, AI Assistant tools (specifically Large Language Models like Google Gemini/ChatGPT) were utilized as an active coding assistant and debugging partner during the development of this project.